In [16]:
```
# 도미 및 빙어 길이와 무게 데이터를 각각 리스트로 저장한다.

bream_length = [
    25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7,
    30.0, 30.0, 30.7, 31.0, 31.0, 31.5, 32.0,
    32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0,
    34.0, 34.5, 35.0, 35.0, 35.0, 35.0, 36.0,
    36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0
]

bream_weight = [
    242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0,
    390.0, 450.0, 500.0, 475.0, 500.0, 500.0, 340.0,
    600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0,
    685.0, 620.0, 680.0, 700.0, 725.0, 720.0, 714.0,
    850.0, 1000.0, 920.0, 955.0, 925.0, 975.0, 950.0
]

smelt_length = [9.8, 10.5, 10.6, 11.0, 11.2, 11.3, 11.8,
    11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0
]

smelt_weight = [
    6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0,
    9.9, 9.8, 12.2, 13.4, 12.2, 19.7, 19.9
]
```

In [17]:
```
# 도미와 빙어의 길이 데이터를 하나의 리스트로 합친다.
# 도미와 빙어의 무게 데이터를 하나의 리스트로 합친다.

length = bream_length + smelt_length
weight = bream_weight + smelt_weight

print (length), print (weight)
print (len(length))
print (len(weight))
```

```
[25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7, 31.0, 31.0, 31.
5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0, 34.0, 34.5, 35.0, 35.0, 3
5.0, 35.0, 36.0, 36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0, 9.8, 10.5, 10.6,
11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0]
[242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0, 450.0, 500.0, 475.
0, 500.0, 500.0, 340.0, 600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0, 68
5.0, 620.0, 680.0, 700.0, 725.0, 720.0, 714.0, 850.0, 1000.0, 920.0, 955.0,
925.0, 975.0, 950.0, 6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2, 13.
4, 12.2, 19.7, 19.9]
49
49
```

In [18]:
```
#데이터의 타깃을
import numpy as np

fish_data = np.column_stack((length,weight))
fish_target = np.concatenate((np.ones(35),np.zeros(14)))
```

```
print(len(length),len(weight))
print(fish_data[15])
print(fish_target)

# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# np.column_stack: 배열들을 열 방향으로 이어 각 샘플의 특성 열을 구성한다.
# np.concatenate: 배열을 지정한 축으로 연결한다. 1차원 배열 연결에도 사용한다.
# np.zeros(n): 값이 0인 배열을 만든다.
# np.ones(n): 값이 1인 배열을 만든다.
```

```
49 49
[ 32. 600.]
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0.]
```

온라인 3주차에서 보충 — `np.vstack()`, `np.full()`

In [19]:

```
# [온라인 3주차 보충] 오프라인에서는 다루지 않은 NumPy 배열 함수 2개
# np.vstack(): 여러 배열을 세로(행) 방향으로 쌓는다.
row_stacked = np.vstack(([1, 2, 3], [4, 5, 6]))
print('vstack result:\n', row_stacked)  # 2행 3열

# np.full((행, 열), 값): 지정한 크기의 배열을 같은 값으로 채운다.
filled = np.full((2, 3), 9)
print('full result:\n', filled)  # 2행 3열, 모든 원소가 9

# 비교: 기존 오프라인 np.column_stack()은 원소를 열 단위로 배치한다.
print('column_stack result:\n', np.column_stack(([1, 2, 3], [4, 5, 6])))
```

```
vstack result:
 [[1 2 3]
 [4 5 6]]
full result:
 [[9 9 9]
 [9 9 9]]
column_stack result:
 [[1 4]
 [2 5]
 [3 6]]
```

In [20]:

```
#훈련데이터와 타깃데이터를 나눈다
from sklearn.model_selection import train_test_split

train_input, test_input, train_target, test_target = train_test_split(
    fish_data,
    fish_target,
    random_state=42
)

print(train_input.shape, test_input.shape)
print(train_target.shape, test_target.shape)
print(train_target)
print(test_target)
```

```
# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# train_test_split(X, y, ...): 입력 X와 정답 y를 같은 인덱스로 분할해 훈련·테스트 세트
# random_state: 데이터 분할의 난수 상태를 고정해 반복 실행 시 같은 분할을 재현한다.
# shape: NumPy 배열의 차원별 크기를 보여준다. 예: (42, 1)은 42행 1열.
```

```
(36, 2) (13, 2)
(36,) (13,)
[1. 1. 1. 0. 0. 0. 1. 1. 1. 1. 1. 1. 1. 0. 1. 1. 1. 1. 0. 1. 1. 1. 0. 0.
 1. 0. 1. 1. 1. 0. 1. 1. 0. 1. 1. 0.]
[1. 0. 0. 0. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

온라인 3주차에서 보충 — `stratify`로 클래스 비율 유지

In [21]:
```
# [온라인 3주차 보충] 분류의 원본 정답(도미=1, 빙어=0) 비율을 고려해 분할한다.
# stratify=fish_target: train/test 세트의 클래스 비율이 전체와 비슷하도록 나눈다.
# 기존 오프라인 예제의 train_input 등을 덮어쓰지 않도록 별도 이름을 사용한다.
(
    strat_train_input, strat_test_input,
    strat_train_target, strat_test_target
) = train_test_split(
    fish_data, fish_target,
    stratify=fish_target,
    random_state=42
)

# 정답이 0과 1이므로 평균은 도미(1) 비율을 뜻한다.
print('전체 도미 비율:', np.mean(fish_target))
print('훈련 도미 비율:', np.mean(strat_train_target))
print('테스트 도미 비율:', np.mean(strat_test_target))
print('분할 후 입력 배열:', strat_train_input.shape, strat_test_input.shape)
```

```
전체 도미 비율: 0.7142857142857143
훈련 도미 비율: 0.7222222222222222
테스트 도미 비율: 0.6923076923076923
분할 후 입력 배열: (36, 2) (13, 2)
```

In [22]:
```
# 최근접 이웃
# 2시 34분 사진에 값넣기

from sklearn.neighbors import KNeighborsClassifier
kn = KNeighborsClassifier()

# 훈련
kn.fit(train_input, train_target)

print(kn.score(train_input, train_target)) #훈련데이터의 점수
print(kn.score(test_input, test_target))    #테스트 데이터의 점수


# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# KNeighborsClassifier: 가까운 k개 이웃의 클래스로 분류하는 모델이며 기본 n_neighbors
# fit(X, y): 입력 데이터와 정답을 바탕으로 모델을 훈련한다.
# score(X, y): 분류에서는 정확도, 회귀에서는 결정계수 R²를 반환한다.
```

```
1.0
1.0
```

In [23]:
```
#샘플평가하기
one = np.array([[25,150]])
distances, idx = kn.kneighbors(one)
print(kn.predict(one), kn.predict_proba(one))
kn.kneighbors(one)  #one과의 이웃으 정보(거리, 인덱스)

# ——— 중간고사 학습 주석 (원본 코드 수정 없음) ———
# predict_proba(X): 클래스별 예측 확률을 반환한다. 열 순서는 모델의 classes_ 순서다.
# predict(X): 학습된 모델이 입력 샘플의 클래스 또는 연속값을 예측한다.
# kneighbors(X): 가까운 이웃까지의 거리와 훈련 세트 내 인덱스를 돌려준다.
# np.array: 파이썬 리스트 등을 NumPy 배열로 변환한다.
```

```
[0.] [[0.6 0.4]]
```

Out[23]:
```
(array([[ 92.00086956, 130.48375378, 138.32150953, 140.00603558,
         140.62090883]]),
 array([[12, 29,  5, 19,  4]]))
```

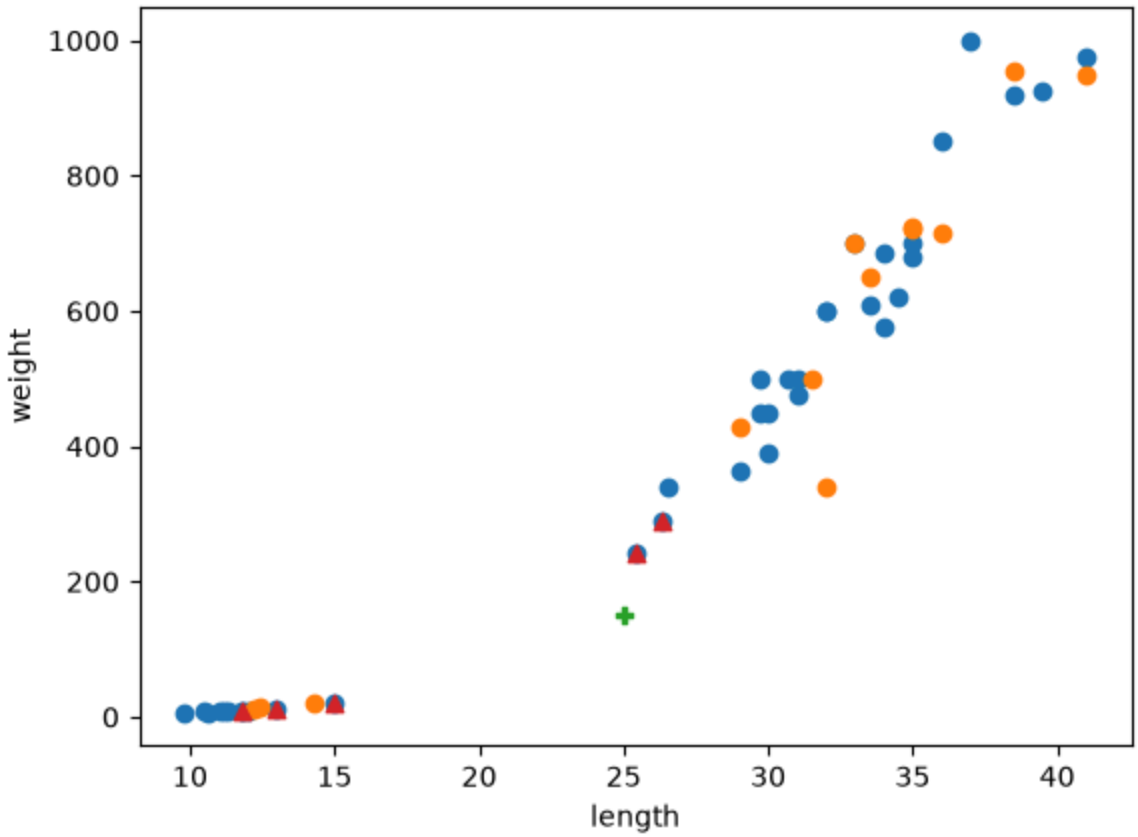
In [24]:
```
#훈련데이터 테스트데이터 샘플 샘플의 이웃을
import matplotlib.pyplot as plt

plt.scatter(train_input[:,0], train_input[:,1])
plt.scatter(test_input[:,0], test_input[:,1])

plt.scatter(one[0,0],one[0,1],marker='P')
plt.scatter(train_input[idx,0], train_input[idx,1], marker='^')

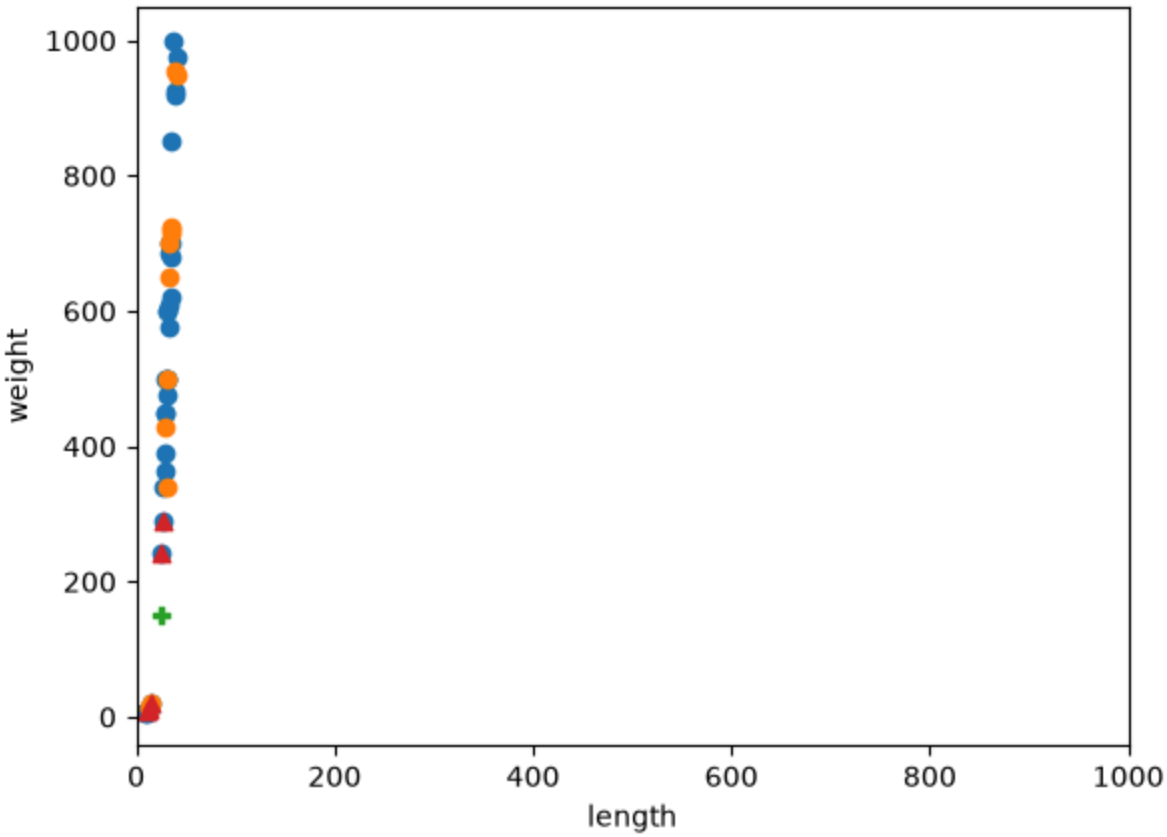
plt.xlabel('length')
plt.ylabel('weight')
plt.show()

# ——— 중간고사 학습 주석 (원본 코드 수정 없음) ———
# plt.scatter(x, y): 실제 데이터/예측 데이터를 산점도로 나타낸다.
# plt.show(): 지금까지 만든 그래프를 출력한다.
```



In [25]:
```
#훈련데이터, 테스트데이터, 샘플, 샘플의 이웃을 산포도로 디스플레이

plt.scatter(train_input[:,0], train_input[:,1])
plt.scatter(test_input[:,0], test_input[:,1])
plt.scatter(one[0,0],one[0,1],marker='P')
plt.scatter(train_input[idx,0], train_input[idx,1], marker='^')
plt.xlim(0,1000)
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```



4주차

In [26]:

```
#훈련데이터의 평균
#axis=0은 열방향 평균, axis=1 행방향 평균
mean = np.mean(train_input, axis=0)
std = np.std(train_input, axis=0)

# 표준화시 타겟은 표준화 하면 안 된다
train_scaled = (train_input - mean) / std
test_scaled = (test_input - mean) / std

one_scaled = (one - mean) / std

print(mean, std)
print(train_scaled)

# ── 중간고사 학습 주석 (원본 코드 수정 없음) ──
# np.mean(..., axis=0): 각 특성 열의 평균을 구할 때 쓴다.
# np.std(..., axis=0): 각 특성 열의 표준편차를 구할 때 쓴다.
```

```
[ 26.175      418.08888889] [ 10.21073441 321.67847023]
[[ 0.37460577  0.09920189]
 [ 0.27666962 -0.17125451]
 [ 0.34522492  0.2546366 ]
 [-1.45680021 -1.27266487]
 [-1.40783213 -1.26862357]
 [-1.29030875 -1.26178444]
 [ 0.57047806  0.56550602]
 [ 0.44316107  0.2546366 ]
 [ 0.66841421  0.87637544]
 [ 0.86428651  0.87637544]
 [ 1.4519034   1.73126635]
 [ 1.20706303  1.56028817]
 [-0.07590051 -0.54740651]
 [-1.3882449  -1.26924531]
 [ 1.30499918  1.57583164]
 [ 0.34522492  0.09920189]
 [ 1.06015881  1.80898371]
 [ 0.47254192  0.2546366 ]
 [-1.53514913 -1.2763953 ]
 [ 0.01224202 -0.39818919]
 [ 0.76635036  0.82974503]
 [ 0.03182925 -0.24275448]
 [-1.52535551 -1.27794965]
 [-1.60370443 -1.27888226]
 [ 0.86428651  0.81420156]
 [-1.46659382 -1.26924531]
 [ 0.47254192  0.17691924]
 [ 0.81531843  0.6276799 ]
 [ 0.71738229  0.59659296]
 [-1.09443646 -1.2378475 ]
 [ 0.76635036  0.48778866]
 [ 0.37460577 -0.08731977]
 [-1.40783213 -1.26893444]
 [ 0.57047806  0.56550602]
 [ 0.96222266  1.34267957]
 [-1.48618105 -1.26955618]]
```

In [27]:
```
kn.fit(train_scaled, train_target)
print(kn.score(train_scaled, train_target))
print(kn.score(test_scaled, test_target))
```

```
1.0
1.0
```

In [28]:
```
print(kn.predict(one_scaled))
#여기 무언가

# ——— 중간고사 학습 주석 (수정본) ———
# [오류 수정] 존재하지 않는 속성 kn.pre를 학습된 모델의 예측 메서드 predict로 바꿨습니다.
```

```
[1.]
```

In [29]:
```
#훈련데이터, 테스트데이터, 샘플, 샘플의 이웃을 산포도로 디스플레이

plt.scatter(train_scaled[:,0], train_scaled[:,1])
plt.scatter(test_scaled[:,0], test_scaled[:,1])
```

```
plt.scatter(one_scaled[0,0],one_scaled[0,1],marker='P')
distances, idx = kn.kneighbors(one_scaled)
plt.scatter(train_scaled[idx,0], train_scaled[idx,1], marker='^')
plt.xlabel('length')
plt.ylabel('weight')
plt.show()

# ── 중간고사 학습 주석 (수정본) ──
# [오류 수정] 새 샘플의 표준화된 이웃을 재계산하고 표준화된 좌표로 표시합니다.
```

